

Отчет по разработке Content-Based рекомендательной системы для платформы Steam

Выполнил: [Шитоев Данил]

11 апреля 2026 г.

Содержание

1	Введение и Постановка задачи	2
2	Описание структуры и анализ данных	2
2.1	Структура данных и контентные признаки	2
2.2	Статистический анализ после предобработки	2
3	Подход к реализации	3
3.1	Векторизация контента (TF-IDF)	3
3.2	Профиль пользователя и Рекомендация	3
4	Демонстрация на конкретных примерах	3
5	Гиперпараметры модели и влияние их изменения	4
5.1	Сводная таблица результатов	4
5.2	Интерпретация влияния гиперпараметров	4
6	Сравнение с KNN и SVD	4
6.1	Анализ сравнения	5
7	Выводы	5

1 Введение и Постановка задачи

Целью данной работы является разработка и оценка качества контентной рекомендательной системы (Content-Based Filtering) на основе текстовых данных платформы Steam. В отличие от коллаборативных методов (KNN, SVD), которые опираются на паттерны поведения пользователей, контентный подход анализирует атрибуты самих объектов (игр) и предпочтения пользователя к этим атрибутам.

Ключевые задачи:

1. Реализовать векторизацию контента игр на основе отзывов пользователей и тегов (TF-IDF).
2. Построить профили пользователей на основе их истории положительных взаимодействий.
3. Провести эксперименты по подбору гиперпараметров векторизатора (`max_features`, `ngram_range`).
4. Оценить качество рекомендаций с использованием метрик ранжирования (Precision, Recall, HitRate, NDCG).
5. Сравнить результаты с ранее реализованными моделями Hybrid KNN и SVD.

2 Описание структуры и анализ данных

Для обучения модели использовался датасет `steam_reco_interactions_clean.csv`.

2.1 Структура данных и контентные признаки

В отличие от предыдущих задач, где использовались только ID и оценки, здесь ключевую роль играют текстовые поля:

- **review_text:** Текст отзыва пользователя. Содержит субъективную оценку, описание геймплея, графики и сюжета.
- **tags_list:** Список тегов, присвоенных игре сообществом (жанры, особенности, настроение). Это структурированный контент, который часто точнее описывает суть игры, чем свободный текст.
- **y_recommended:** Бинарная метка (1 — рекомендация). Использовались только положительные отзывы для формирования контента, так как мы хотим рекомендовать то, что нравится пользователям.

2.2 Статистический анализ после предобработки

- **Количество игр с контентом:** 351 уникальная игра.
- **Пользователей в тестовой выборке:** 1511.
- **Обработка текста:** Приведение к нижнему регистру, удаление пунктуации. Теги дублировались в итоговом документе для повышения их веса при векторизации.
- **Агрегация:** Все отзывы и теги для одной игры объединялись в один “супер-документ”, представляющий контент этой игры.

3 Подход к реализации

Реализация базировалась на классическом подходе TF-IDF (Term Frequency-Inverse Document Frequency) и косинусном сходстве.

3.1 Векторизация контента (TF-IDF)

Для каждой игры строился вектор признаков. Использовался `TfidfVectorizer` из библиотеки `scikit-learn` со следующими параметрами:

- `stop_words='english'`: Удаление стоп-слов английского языка.
- `dtype=np.float32`: Оптимизация памяти.
- `min_df=2`, `max_df=0.95`: Отсечение слишком редких (шум) и слишком частых (неинформативных) терминов.

Итоговый документ для игры формировался как конкатенация:

$$Doc_{game} = Tags + Tags + Review_{text}$$

Дублирование тегов усиливает их вклад в итоговый вектор, так как жанровая принадлежность часто важнее отдельных слов из отзыва.

3.2 Профиль пользователя и Рекомендация

1. **Профиль пользователя:** Формируется как центроид (среднее арифметическое) TF-IDF векторов всех игр, которые пользователь рекомендовал в обучающей выборке.

$$P_u = \frac{1}{|I_u|} \sum_{i \in I_u} V_i$$

где V_i — вектор игры i , I_u — множество игр пользователя u .

2. **Скоринг:** Для каждой игры вычисляется косинусное сходство между профилем пользователя и вектором игры:

$$Score(u, i) = \cos(P_u, V_i) = \frac{P_u \cdot V_i}{\|P_u\| \|V_i\|}$$

3. **Фильтрация:** Игры, которые пользователь уже оценивал, исключаются из выдачи.

4 Демонстрация на конкретных примерах

Для лучшей конфигурации модели (`max_features=5000`, `ngram_range=(1,1)`) были получены следующие рекомендации для пользователя `12536475869trhg sdf`:

Интерпретация: Все рекомендованные игры относятся к жанрам MMO-Shooter или Sci-Fi Action. Это указывает на то, что профиль пользователя успешно уловил его предпочтение к многопользовательским научной-фантастическим шутерам на основе тегов и лексики отзывов.

Таблица 1: Пример рекомендаций Content-Based модели

Rank	Game ID	Game Name	Score
1	17080	Tribes: Ascend	0.5745
2	207230	ArcheBlade™	0.5471
3	17020	Global Agenda: Free Agent	0.5330
4	218230	PlanetSide 2	0.5266
5	9900	Star Trek Online	0.5003

5 Гиперпараметры модели и влияние их изменения

Были протестированы различные комбинации параметров векторизатора:

- `max_features`: 5000, 10000, 20000.
- `ngram_range`: (1, 1) — униграммы, (1, 2) — униграммы и биграммы.

5.1 Сводная таблица результатов

Таблица 2: Результаты экспериментов с гиперпараметрами

<code>max_features</code>	<code>ngram_range</code>	HitRate@10	Precision@10	Recall@10	NDCG@10
5000	(1, 1)	0.1046	0.0105	0.1042	0.0531
10000	(1, 1)	0.1046	0.0105	0.1042	0.0533
10000	(1, 2)	0.1006	0.0101	0.1003	0.0505
20000	(1, 2)	0.1013	0.0101	0.1009	0.0507

5.2 Интерпретация влияния гиперпараметров

1. **Влияние n-gramms:** Использование биграмм (`ngram_range=(1,2)`) не улучшило, а даже слегка ухудшило качество метрик.

- *Причина:* Теги в Steam обычно состоят из одного или двух слов, но часто разделены запятыми. При очистке текста биграммы могли захватывать бессмысленные сочетания слов из разных предложений отзывов, добавляя шум в модель. Униграммы оказались более устойчивыми.

2. **Влияние max_features:** Увеличение словаря с 5000 до 20000 не дало существенного прироста качества.

- *Причина:* Основные жанровые признаки и ключевые слова содержатся в топ-5000 наиболее важных терминах. Дальнейшее расширение словаря добавляет редкие специфические слова, которые мало влияют на общее сходство.

Лучшая конфигурация: `max_features=5000`, `ngram_range=(1, 1)` (или 10000, результаты идентичны по HitRate).

6 Сравнение с KNN и SVD

Для объективной оценки эффективности Content-Based подхода сравним его лучшие результаты с лучшими результатами моделей Hybrid KNN и SVD, полученными в предыдущих заданиях.

Таблица 3: Сравнение лучших моделей (HitRate@10, Precision@10, NDCG@10)

Модель	HitRate@10	Precision@10	NDCG@10
Hybrid KNN	0.1781	0.0178	0.0948
SVD (FunkSVD)	0.0695	0.0069	0.0297
Content-Based	0.1046	0.0105	0.0531

6.1 Анализ сравнения

1. **Content-Based vs SVD:** Контентная модель значительно превосходит SVD по всем метрикам (HitRate выше в 1.5 раза, NDCG почти в 2 раза).

- *Причина:* SVD страдал от сильной разреженности данных и переобучения при увеличении числа факторов. Content-Based модель использует плотные векторы признаков (теги/текст), которые есть у каждой игры, что делает её более устойчивой на малых данных.

2. **Content-Based vs Hybrid KNN:** Гибридная KNN модель показывает наилучшие результаты (HitRate 0.178 против 0.105 у CB).

- *Причина:* KNN использует прямую историю взаимодействий “похожих людей”. Если пользователь А и пользователь Б лайкнули одну и ту же нишевую игру, KNN сразу свяжет их. Content-Based модели нужно, чтобы эта нишевая игра имела похожие теги/отзывы с другими играми пользователя, что не всегда очевидно (например, разные игры могут быть “смешными” по разным причинам).
- Однако, разрыв не критический. Content-Based показывает достойное качество, не используя информацию о других пользователях.

7 Выводы

1. **Эффективность контентного подхода:** Модель на основе TF-IDF показала себя устойчивее, чем матричная факторизация (SVD), на данном датасете. Это связано с тем, что текстовые признаки (теги) являются плотными и информативными, в то время как матрица взаимодействий крайне разрежена.
2. **Оптимальные параметры:** Использование простых униграмм (`ngram_range=(1,1)`) и ограниченного словаря (`max_features=5000`) оказалось достаточным. Усложнение модели биграммами привело к добавлению шума.
3. **Сравнительное положение:**
 - **KNN** — лидер по точности, так как лучше всего ловит социальные паттерны и скрытые ассоциации.
 - **Content-Based** — уверенный “середняк”. Он хуже KNN, но лучше SVD. Его главное преимущество — способность рекомендовать **новые игры** (Cold Start Item), которых нет в матрице взаимодействий, но есть описание.
 - **SVD** — показал наименьшее качество на ранних этапах обучения из-за переобучения.

4. **Рекомендации по применению:** В реальной системе Steam оптимально использовать **гибридный подход**: использовать KNN/SVD для основного ранжирования популярных игр, а Content-Based модель — для заполнения пробелов (новые релизы, нишевые инди-игры) и объяснения рекомендаций (“потому что вы любите жанр RPG”).